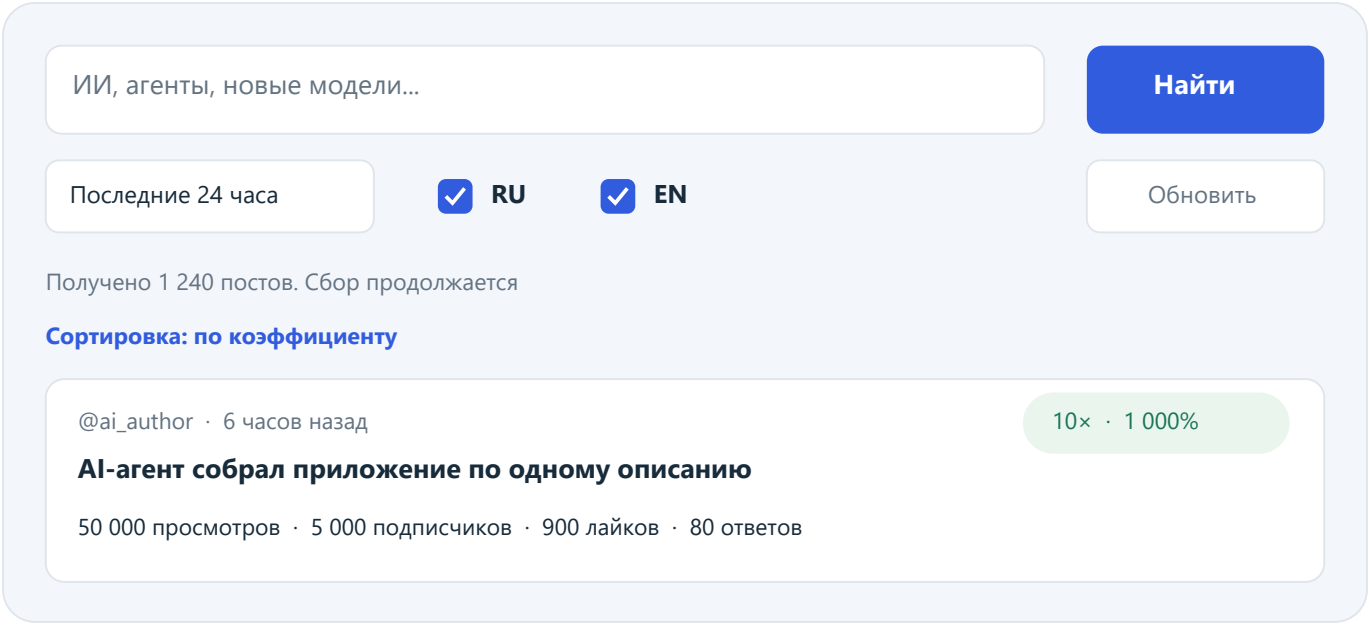
Поиск AI-постов в X

Женя вводит тему и получает посты с высоким числом просмотров относительно подписчиков автора.

Первая версия | По запросу | Без платных API

Одна страница: поиск, фильтры, результаты



Условный пример интерфейса и цифр.

RU и EN - отдельные галочки. По умолчанию включены обе; хотя бы одна должна оставаться. Язык влияет на поиск, а сортировка меняет порядок результатов.

Что показываем у поста

Текст и ссылку, автора, дату и возраст поста, просмотры, подписчиков, лайки, ответы, коэффициент и процент.

Как сортируем

По коэффициенту, просмотрам, дате, лайкам или ответам. Сортировка работает по всей загруженной выдаче.

Коэффициент = просмотры / подписчики

50 000 / 5 000 = 10× | В процентах: 1 000%

Это отношение просмотров к подписчикам. Оно не измеряет уникальный охват или скорость роста. Период поиска относится к дате публикации.

Встраиваем поиск в Tuber

Работаем в том же репозитории и используем общую базу Tuber. Добавляем веб-страницу и поиск по запросу пользователя.

Веб-страница
поиск и выдача

>

Модуль X
сбор через twscrape

>

База Tuber
посты, авторы, метрики

1 Подключаем бесплатный сбор через twscrape

Добавляем его в модуль X вместо Nitter для нового поиска. Сначала проверяем поиск, просмотры и подписчиков на нашем аккаунте. Сессия X хранится на сервере; в браузер и Git её не передаём.

2 Поиск сразу сохраняет посты

В текущем discover.py сохраняются кандидаты-авторы. Новый поиск сразу пишет публикации, без ожидания проверки автора. Собираем всю доступную выдачу по страницам в режиме Latest, без своего предела в 50 или 500 постов.

3 Загружаем и обрабатываем одновременно

При старте фиксируем границы периода. RU и EN - две ветки в общей очереди аккаунта, с учётом его лимитов. Пока загружается следующая страница, предыдущую сохраняем, считаем коэффициент и показываем. Страницы каждой ветки идут по порядку.

4 Переиспользуем хранилище и данные авторов

Авторы и подписчики идут в source, посты - в content, замеры - в metric_snapshot и content_latest. Сохраняем подписчиков из ответа с постом: текущий collect.py их теряет при догрузке метрик.

Что добавить в проект

Небольшой веб-слой на Python и страницу HTML/JS. В общей БД - задачи поиска и связи с найденными постами: запрос, языки, период, статус и позиция загрузки каждой ветки. Статус и позицию фиксируем после сохранения страницы, чтобы продолжать после сбоя.

Для темы ИИ задаём варианты на обоих языках: ИИ, искусственный интеллект, AI, artificial intelligence. Галочки языка сами запрос не переводят.

Сохраняем и используем повторно

Кэш - уже полученные данные. Он общий для поисков и задач Tuber. Ниже - начальные сроки; их можно менять в настройках.

Что сохраняем	Как используем
Подписчики автора 24 часа	Ключ кэша - ID автора X. Подписчиков из ответа с постом сохраняем со временем замера. Иначе берём кэш не старше 24 часов. Отдельный запрос нужен только без свежих данных. Проверка срока и защита от одновременных запросов общие с текущей задачей followers; успешный запрос повторно в эти 24 часа не делаем.
Готовая выдача 5 минут	Одинаковый поиск повторно показываем из базы. Учитываем запрос, языки, фильтры и период; сортировка в ключ кэша не входит. Для «24 часов» повторно используем зафиксированное окно и показываем его границы и время сбора. Наличие постов в базе само по себе не означает, что поиск завершён.
Посты и метрики По ID и времени	Один пост храним один раз по ID X; разные посты одного автора оставляем. Каждый новый ответ обновляет метрики и время замера. Старые цифры показываем с датой обновления; суточный кэш подписчиков не замораживает просмотры поста.

Что происходит при повторном действии

Сортировка или EN + RU → RU: используем уже собранные данные. **RU → EN + RU:** догружаем EN за тот же период. **Новая тема или период:** новый поиск, с повторным использованием постов и авторов по ID.

«Обновить»: обходим кэш готовой выдачи и запускаем актуальный поиск; для «24 часов» фиксируем новое окно. Кэш авторов сохраняется. Если тот же поиск уже идёт, подключаемся к нему; закрытие страницы не отменяет задачу.

Когда результат можно считать готовым

Все языковые ветки завершены штатно. Лимит X, сбой или зацикленный курсор означают неполную выдачу: сохраняем прогресс и продолжаем после восстановления. Нет данных или подписчиков 0 - коэффициент недоступен. Репост объединяем с оригиналом; цитату считаем по её автору и метрикам.

twscrape · Tuber · Полнота выдачи зависит от X. Фоновое наблюдение и уведомления - следующий этап.